

PROJECT MANAGEMENT IS

Sprint 2 report

Project: AI Student Request Routing and Support System

Increment: Human review, staff operations, and student status tracking

Duration: Two weeks, as required by the lab sheet

Source scope: US3, US4, and US5, 15 total source effort points

Readiness: Implemented combined Sprint 1-2 increment; see verification evidence

Human acceptance: Not recorded; actual team/teacher acceptance must be recorded after the defence

Sprint goal

Keep uncertain requests in a visible human review workflow, let authenticated support staff correct routes and resolve requests, and let students follow their own request status.

Planning and selected backlog

Story	Source effort	Owner role	Dependency	Completion evidence
US3. Low-confidence escalation	5	AI/Backend Developer	US1, confidence score, review storage	Configurable threshold, review queue, correction log tests
US4. Staff manual routing dashboard	6	Frontend Developer and UX Designer	Support account, US3	Role restriction, filtering/sorting, reassignment tests
US5. Request status tracking	4	Full-stack Developer	US1, request status model	Owner isolation, timestamp, resolution/status tests

The source deadlines are 15.12.2026, 22.12.2026, and 29.12.2026. They are preserved in the product backlog. Their span exceeds a single two-week sprint; course dates govern the actual lab schedule. A seven-sprint roadmap reconciles the source's four feature groups with the course's seven iterations.

Increment delivered

- OpenAI/local mode visibility, provider explanation/version, and persisted confidence threshold (default 0.65).
- Low-confidence requests stored with status Escalated and no department assignment.
- Four-business-hour review deadlines, computed for Mon-Fri 09:00-17:00 at UTC+5.
- Overdue review warnings, with no claim that software alone guarantees staff attendance.
- SDU helpdesk (helpdesk@sdu.edu.kz) support-only queue: oldest first, original text, suggestion, confidence and route.
- Filtering by department, status, open/resolved state, and manually set priority.

- Direct department selection and Route button, supporting two-action manual routing from a visible card.
- Reassignment that changes the destination queue, resets to Received, and notifies the student.
- Staff routing decisions logged with message snapshot and reviewer for later retraining.
- Received -> In Progress -> Resolved status workflow, resolution explanation, reopen, and event history.
- Student list status updates through 15-second polling while the portal is active, preserving any typed request text.
- Transactional version checks that reject conflicting simultaneous staff decisions.

Acceptance and testing

OpenAI failures preserve requests in staff review. API keys are configured locally; live verification remains pending. The backend suite checks escalation, administrative persistence, manual routing, correction logs, queue ordering/filters, role restrictions, 500-request retrieval, status/timestamp propagation, another student's isolation, resolution/reopen, and optimistic concurrency.

Student status refreshes every 15 seconds in an active tab while preserving form text. Background tabs pause polling; open details remain a snapshot until reopened. The one-minute target assumes an available server/network. Actual browser timing is in docs/evidence/verification.json.

Sprint review / demo

1. Start the default Sprint 2 app and submit an ambiguous request as a student.
2. Sign in as support; show the escalation and original AI suggestion/confidence.
3. Select Academic Advising and press Route directly on the card.
4. Show the changed queue, then start work and resolve a routed request.
5. Sign back in as the student; show the status, resolution, and history.
6. Sign in as the second student and demonstrate that the first student's requests are hidden.
7. Sign in as the administrator; update the threshold and inspect correction summaries.
8. Show the criteria audit, test results, and Sprint 2 board/backlog.

Technical retrospective

What worked: explicit support-only permissions and owner-filtered reads protect request privacy. A version field plus a write transaction prevents staff decisions from overwriting one another. Read-only refresh avoids duplicate notices.

Challenges: human review depends on staffing, and institutional privacy approval is external. Review deadlines expose overdue work; the synthetic model needs broader validation.

Action items: confirm staffing, business hours, a holiday calendar, and privacy policy; collect authorized labels; implement US6-US10 later. The team must record its actual review and retrospective outcomes, and verify the external model with its own API account.

Current boundary

US6-US10 remain in the product backlog. This increment stores manual priority, preserved non-English input, and staff labels. Automatic urgency detection, multilingual classification/translation, feedback, analytics, and retraining are scheduled for later sprints.